

BAKI

Security Hardening & Secure Development Guide

Internal Team Documentation

Version 1.0 | 13 August 2026

A practical security reference for the Baki surplus-food matching platform, covering the hardening work completed so far, remaining controls, secure coding expectations, testing, deployment, privacy, and incident response.

Prepared for the Baki development team

1. Security Hardening Status

This guide records the security-hardening work being carried out on Baki. The status below reflects the work identified in the current hardening plan and the implementation details visible in the supplied Baki templates.

Control	Status	Team note
Input validation	DONE	Implemented as a completed hardening item.
Debug mode disabled	DONE	Production application must not run with Flask debug mode enabled.
Secure HTTP headers	DONE	Security headers have been committed and should be regression-tested.
CSP implementation	NEXT / IN PROGRESS	Must be aligned with the current JavaScript and external resources.
Remove unsafe inline JS	PENDING	Current templates contain inline JavaScript and need migration to external scripts/nonces/hashes.
Secure Jinja rendering	PENDING	Review uses of rendered data, especially data passed into JavaScript.
CSRF protection	PENDING	POST forms should have server-side CSRF protection.
Secure cookies / sessions	PENDING	Review cookie flags and session configuration before production.
Error handling	PENDING	Avoid exposing stack traces or sensitive implementation details.
Dependency/security updates	ONGOING	Keep Flask and dependencies patched and periodically reviewed.
Security testing & documentation	ONGOING	Use this guide as the team reference and update the change log.

2. Purpose and Security Principles

Baki accepts information from vendors and subscribers, processes listings, matches surplus food to organizations, and exposes selected listing information on a public community board. Security controls therefore need to protect both the application and the people whose information it processes.

Core principles

- Treat all browser-submitted data as untrusted until validated on the server.
- Keep production debug mode disabled.
- Apply secure HTTP response headers consistently.
- Use a restrictive Content Security Policy appropriate to the application.
- Avoid unsafe inline JavaScript and unnecessary use of Jinja's `|safe` filter.
- Protect every state-changing POST action against CSRF.
- Do not expose secrets, credentials, stack traces, or internal configuration to users.
- Minimize the personal information exposed on public pages.

3. Baki Data & Privacy Considerations

The supplied templates show that Baki handles several categories of information. The team should explicitly decide which data is public, private, or restricted.

Data	Current use visible in templates	Security/privacy consideration
Vendor/business name	Displayed on listings/results	Treat as user-controlled input; escape when rendered.

Data	Current use visible in templates	Security/privacy consideration
Vendor contact number	Displayed publicly on the live board	This is a deliberate public exposure decision and should be documented and reviewed.
Subscriber name	Collected by /subscribe	Should remain private and only be used for notifications.
Subscriber phone/WhatsApp	Collected by /subscribe	Sensitive contact data; do not expose on the public board.
Food type / quantity	Listing and matching	Validate accepted values and numeric ranges.
Country / location	Listing and subscription matching	Validate against server-side allowed values.
Sale price	Community-sale listing	Validate format and length; do not trust client-side validation alone.
NGO/matching information	Shown on result pages	Avoid exposing internal identifiers or private contact details.

4. Current Template Security Review

The following observations are based specifically on the homepage, browse.html, and result.html code supplied for this project.

Public contact information

browse.html renders the vendor contact directly on the public board. The homepage also tells vendors that their number will be shown publicly. This should remain an intentional product/privacy decision, with a future option to use an in-app contact mechanism or masked contact details if the team decides public phone numbers are too risky.

Jinja data inserted into JavaScript

The homepage currently contains `const LOCATIONS_BY_COUNTRY = {{ locations_json | safe }}`. Because this crosses the template-to-JavaScript boundary, the value must be generated as valid JSON and handled safely. The team should avoid using `|safe` for arbitrary user-controlled strings and should prefer a safe JSON serialization mechanism where available.

Inline JavaScript

The supplied templates contain several inline script blocks. This is important for the CSP work because a strict CSP commonly blocks inline scripts unless a nonce/hash or another deliberate mechanism is used. The preferred direction is to move application JavaScript into static JS files and keep CSP restrictive.

External resources

The templates load Google Fonts from `fonts.googleapis.com`. The final CSP must explicitly account for any external resources the application genuinely needs, while avoiding unnecessary third-party origins.

5. Secure Coding Rules

Before merging application changes, developers should confirm:

- All form values are validated server-side for type, length, range, and allowed values.
- HTML output is escaped by default; trusted HTML is rare and justified.
- User-controlled values are never passed through Jinja `|safe` without a documented reason.
- JavaScript does not construct executable HTML from untrusted values.
- State-changing actions use POST and have CSRF protection.
- Secrets are stored outside source code and are not committed to Git.
- Error responses do not reveal stack traces, credentials, SQL, filesystem paths, or internal configuration.

- Authentication and authorization checks are enforced server-side.
- Security-sensitive changes are tested before deployment.
- Dependencies are reviewed and updated regularly.

6. Security Testing Checklist

- Submit empty, oversized, malformed, and unexpected values to every form.
- Test numeric fields with negative, zero, very large, decimal, and non-numeric input.
- Test listing_type with values other than donate/sell.
- Test country/location values that are not present in the server-side data.
- Test sale_price with HTML, JavaScript-looking strings, excessive length, and unexpected characters.
- Test vendor_name and food_type for reflected/stored XSS payloads.
- Verify CSRF rejection when a valid POST is sent without a valid CSRF token.
- Inspect response headers and confirm the intended CSP and other security headers are present.
- Confirm production error pages do not expose debug information.
- Verify cookies use appropriate Secure, HttpOnly, and SameSite settings where applicable.
- Check that subscriber contact information is never rendered on public pages.
- Test the WhatsApp share feature to ensure untrusted values cannot alter the intended URL structure.

7. Secure Deployment Checklist

- FLASK_ENV / production configuration is appropriate and debug mode is disabled.
- Application secrets are supplied through a secure environment/secret-management mechanism.
- HTTPS is enabled for the public application.
- Security headers are enabled and tested in production.
- CSP is enabled and reviewed after every major frontend change.
- Database credentials are not present in templates or committed source files.
- Logs do not contain passwords, tokens, or unnecessary personal information.
- Backups and recovery procedures are defined.
- Dependencies are pinned/reviewed and patched for known vulnerabilities.
- A rollback procedure exists for failed security or application deployments.

8. Incident Response

If a security problem is discovered, the team should preserve evidence, limit further exposure, fix the cause, and document what happened.

1. **Identify:** Record what was observed, when it was observed, and which component is affected.
2. **Contain:** Disable or restrict the affected functionality if necessary to prevent further damage.
3. **Protect:** Rotate exposed credentials, tokens, or secrets immediately where applicable.
4. **Investigate:** Review application logs, recent changes, affected records, and attack paths.
5. **Remediate:** Fix the root cause and add a regression test so the issue does not return.
6. **Verify:** Retest the affected area and confirm the fix in a controlled environment.

7. **Document:** Record impact, timeline, actions, lessons learned, and follow-up tasks.

9. Team Responsibilities

Area	Primary responsibility
Application code	Developers review validation, output encoding, CSRF, authorization, and secure coding practices.
Dependencies	Assigned team member monitors vulnerable packages and coordinates updates.
Secrets	Only authorized team members manage production secrets; never commit them to source control.
Deployment	Deployment owner confirms production security checklist before release.
Testing	Team tests security controls after changes and records failures/resolutions.
Documentation	Security owner updates this guide and the change log as controls evolve.
Incident response	Team follows the incident procedure and escalates significant incidents promptly.

10. Security Change Log

Date	Change	Status
13 Aug 2026	Input validation hardening	Completed
13 Aug 2026	Production debug mode disabled	Completed
13 Aug 2026	Secure HTTP headers implemented/committed	Completed
13 Aug 2026	CSP / inline JavaScript review	Next / In progress
13 Aug 2026	Secure Jinja rendering review	Pending
13 Aug 2026	CSRF protection review	Pending
13 Aug 2026	Team security documentation	Completed — this guide

11. Important Note for the Team

This document is a working security guide, not a certification or guarantee that Baki is secure. Security controls must be re-tested whenever application behavior, dependencies, infrastructure, authentication, forms, or data flows change. Any control marked pending should be completed and verified before the application is considered production-ready.

End of document